SOFTWARE ENGINEERING LAB – 3

Component Modelling & Architectural Pattern Selection

Problem Statement	4 — Student Project Portfolio & Showcase App
Student	Kirthana S | PES1UG24AM136
Section / Semester	C | 5

1. Problem Overview

The Student Project Portfolio & Showcase App allows student teams to submit structured project profiles for the annual showcase. This report focuses on UC-01 — Create Showcase Profile and FR-001/FR-002: capturing required project details and validating the GitHub repository URL before publication.

2. UML Component Diagram

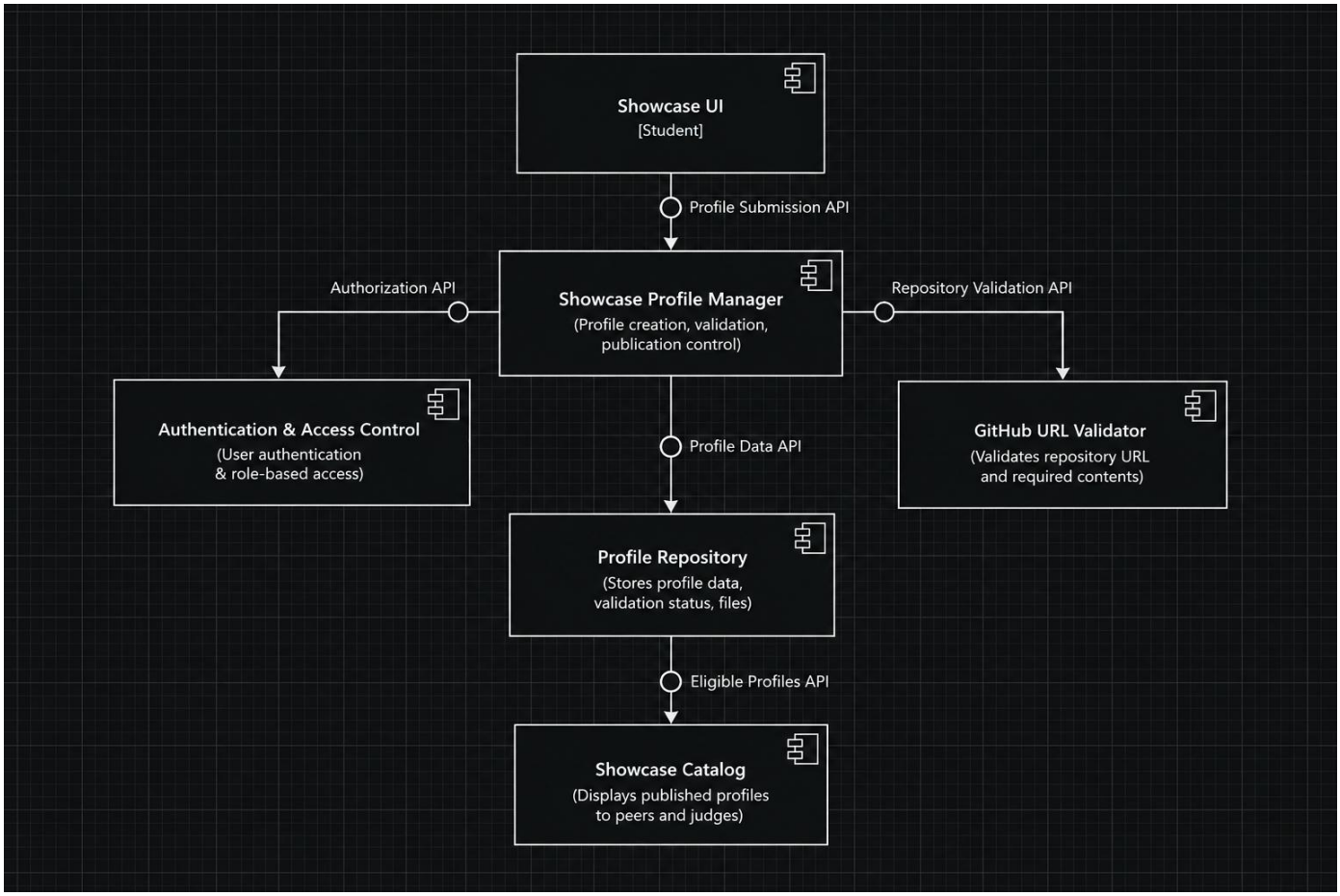


Figure 1. Component model for Showcase Profile Management, showing labelled interfaces and the profile submission/validation data flow.

3. Component Responsibilities

- **Showcase UI:** Collects the project title, team details, abstract, demo video link, and repository URL.
- **Showcase Profile Manager:** Coordinates mandatory-field checks, validation, profile creation, and publication eligibility.
- **Authentication & Access Control:** Checks the student's identity and permissions before profile operations.
- **GitHub URL Validator:** Checks whether the submitted repository URL is valid/verifiable before publication.
- **Profile Repository:** Stores the submitted profile information and repository-validation status.
- **Showcase Catalog:** Displays profiles marked eligible for publication.

4. Architectural Style Selection and Justification

Layered Architecture is selected because the profile workflow separates presentation, application coordination, validation/security, and data persistence.

Reason 1 — Clear separation of responsibilities: The Showcase UI collects the project title, team details, abstract summary, demo video link, and GitHub repository URL. The Showcase Profile Manager coordinates mandatory-field checks and the creation workflow, while the Profile Repository handles persistence. Separating these responsibilities makes the profile workflow easier to maintain and test.

Reason 2 — Controlled repository validation and publication: The GitHub URL Validator checks the submitted repository URL before the profile can become eligible for publication. If the URL is invalid or cannot be verified, the system can return an error and prevent publication until the student corrects and resubmits it. This directly supports FR-002 and the alternate flow in UC-01.

5. Security and Performance Considerations

Security Advantage: Authentication & Access Control centralizes identity and permission checks before profile changes are accepted. Repository validation also prevents malformed or unverifiable repository links from being published as valid project references.

Performance Benefit: The UI, validation, profile-management, and persistence responsibilities are separated behind labelled interfaces. This keeps presentation logic separate from data operations, makes bottlenecks easier to isolate, and supports focused testing without introducing unnecessary distributed-service overhead for this workflow.

6. Conclusion

The component model supports UC-01 by separating profile entry, repository validation, access checks, persistence, and publication eligibility. The labelled interfaces clarify component interactions and provide a maintainable structure aligned with FR-001 and FR-002.